Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Ingeniería de Software

1. Introducción y Objetivos

El diseño de la arquitectura de un sistema de software es una de las decisiones más críticas y de mayor impacto en su ciclo de vida. A medida que los sistemas evolucionan en complejidad y escala, la necesidad de estructuras que promuevan el desacoplamiento, la mantenibilidad y la evolución se vuelve primordial. La industria ha desarrollado diversos estilos arquitectónicos para abordar estos desafíos, cada uno con un conjunto único de principios, fortalezas y compromisos (trade-offs)[1].

Este seminario se centra en el estudio y análisis comparativo de cinco estilos arquitectónicos fundamentales que, aunque comparten el concepto de çapas", proponen formas radicalmente diferentes de organizar las dependencias y aislar la lógica de negocio. Los estilos a analizar son:

- **Arquitectura N-Capas (Tradicional):** El estilo más común y a menudo el punto de partida por defecto. Organiza el sistema en capas horizontales (Presentación, Negocio, Datos), con dependencias que fluyen de arriba hacia abajo. Su simplicidad es su mayor atractivo, pero también su mayor debilidad.
- **Arquitectura Hexagonal (Puertos y Adaptadores):** Propuesta por Alistair Cockburn, este estilo se centra en aislar la lógica de la aplicación (el "hexágono") de sus interacciones con el exterior (bases de datos, interfaces de usuario, servicios). Utiliza "puertos"(interfaces) y .ªdaptadores"(implementaciones) para lograr esta simetría entre el interior y el exterior.
- **Arquitectura Limpia (Clean Architecture):** Popularizada por Robert C. Martin, sintetiza ideas de la Arquitectura Hexagonal y otras. Su principio fundamental es la *Regla de Dependencia*: las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las capas de mayor nivel (políticas y entidades). Esto hace que los detalles (UI, BD) sean "plugins"del núcleo de negocio.
- **Arquitectura Cebolla (Onion Architecture):** Propuesta por Jeffrey Palermo, comparte el objetivo central de la Arquitectura Limpia. Se representa como capas concéntricas donde el Modelo de Dominio es el centro. Todas las dependencias apuntan hacia el centro, y las capas externas (Infraestructura, UI) implementan interfaces definidas en las capas internas.
- **Domain-Driven Design (DDD) como Guía Arquitectónica:** Más que un estilo arquitectónico en sí mismo, DDD es una filosofía de diseño que enfatiza la colaboración con expertos del dominio y la creación de un *Lenguaje Ubicuo*. Conceptos como el *Bounded*

[1] En ingeniería de software, un *trade-off* o compensación es la decisión de sacrificar o disminuir una cualidad o atributo de un sistema (por ejemplo, el rendimiento) para obtener o mejorar otro (por ejemplo, la mantenibilidad).

Context, las *Entidades*, los *Objetos de Valor* y los *Agregados* influyen directamente en cómo se estructuran las arquitecturas orientadas al dominio como Onion y Clean.

La selección no es arbitraria. La Arquitectura N-Capas representa el statu quo del que muchos de estos nuevos estilos intentan escapar. La Arquitectura Hexagonal, Limpia y Onion representan una evolución hacia el desacoplamiento extremo y la independencia de la lógica de negocio. DDD, por su parte, proporciona el "porquéτ el "qué"del núcleo de negocio que estas arquitecturas intentan proteger. Juntas, forman un cuerpo de conocimiento esencial para cualquier arquitecto de software moderno.

Al finalizar el seminario, el estudiante debe ser capaz de analizar, comparar y evaluar críticamente los estilos arquitectónicos N-Capas, Hexagonal, Limpia, Cebolla y Domain-Driven Design (DDD), para justificar la selección de una arquitectura adecuada que resuelva problemas de desacoplamiento, mantenibilidad y evolución en un sistema de software complejo, aplicando los principios de inversión de dependencias y diseño guiado por el dominio.

2. Preguntas

Las siguientes preguntas están diseñadas para que los estudiantes apliquen los conceptos teóricos de los materiales proporcionados, fomentando el análisis crítico y la capacidad de resolver problemas de diseño arquitectónico. Las respuestas deben basarse en la bibliografía orientada para el seminario.

1. **Análisis de Trade-offs (Arquitectura N-Capas vs. Clean/Onion/Hexagonal):** Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un "monolito" difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

   Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, explique detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la **Regla de Dependencia** (Martin) o el uso de **Puertos y Adaptadores** (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

2. **Comparación de Estilos: La Simetría del Hexágono vs. la Jerarquía de las Capas:** La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes (un hexágono simétrico frente a capas concéntricas).

   Analice en profundidad qué problema conceptual resuelve la **simetría** de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva. ¿Cómo se traduce el concepto de "Puerto y Adaptador" en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

3. **Aplicación de DDD y Bounded Context en un Dominio Complejo:** Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones (ej: "póliza"

para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para un ajustador es un número de referencia).

Basado en los conceptos de **Domain-Driven Design** (Evans) y **Bounded Context** (Richards & Ford), explique cómo dividiría este sistema en varios contextos delimitados. ¿Cómo evitaría que el modelo de "Póliza" se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada? Relacione esta solución con la implementación de una **Arquitectura Orientada al Dominio** (como Onion o Clean) y la creación de un **Adaptador de Servicio** para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

4. **Evaluación de la Inversión de Dependencias en la Práctica:** Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase `OrderProcessor` que, para completar su tarea, necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: `OrderProcessor` instancia y llama directamente a la clase `AddressValidatorService`, que a su vez depende de una librería HTTP específica para comunicarse con el servicio externo.

   Con base en los principios de la **Inversión de Dependencias** (Martin) y la **Regla de Dependencia** (Martin), analice y responda:

   a) ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.

   b) Rediseñe la interacción entre `OrderProcessor` y `AddressValidatorService` para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que "los detalles deben depender de las abstracciones".

   c) Explique cómo este nuevo diseño permite que la lógica de negocio (`OrderProcessor`) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo, y sin depender de la librería HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

5. **Análisis de un Problema Real: El "Kitty Problem" y el Principio de Abierto/Cerrado:** En su libro *Fundamentals of Software Architecture*, Richards y Ford presentan el "Kitty Problem" para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal (como la entrega de gatitos) afecta a múltiples servicios previamente existentes (TaxiUI, TaxiFinder, TaxiSelector, etc.), requiriendo cambios coordinados en todos ellos.

   ¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales (como `TaxiFinder` o `TaxiSelector`) llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio? Proponga una solución utilizando componentes como `Kittens` y `Rides` que se añadan como plugins, explicando cómo esto respeta el **Principio de Abierto/Cerrado (OCP)** y qué papel juegan los **Adaptadores** en esta extensión.

6. **Evaluación de la Arquitectura N-Capas frente a Requisitos de Atributos de Calidad:** La Arquitectura N-Capas es criticada por su bajo rendimiento en características

como “Agilidad general”, “Facilidad de despliegue” y “Escalabilidad” (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (open layers) y técnicas como el “fast-lane reader”, se pueden mitigar estos problemas.

Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de **Aislamiento de Capas** y a la mantenibilidad a largo plazo? ¿Por qué la solución de capas abiertas puede llevar a un “sinkhole anti-pattern” y qué dice la regla del 80/20 sobre esto? Finalmente, explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios de DDD y desacoplamiento, son inherentemente más adecuadas que una Arquitectura N-Capas, incluso con optimizaciones.

3. Orientaciones Generales

Para garantizar la correcta entrega y evaluación del seminario, todos los equipos deben cumplir con las siguientes indicaciones.

Formato y Presentación del Documento

El documento final del seminario debe cumplir con los siguientes requisitos de formato:

- **Página de presentación:** El documento debe incluir una página de presentación (portada) que contenga de forma clara y explícita la siguiente información:
  1. Nombre de la actividad: “Seminario de Arquitecturas de Software:Un Análisis Comparativo de Estilos Basados en Capas y Dominio”.
  2. Nombre del equipo.
  3. Nombres completos de todos los integrantes del equipo.
  4. Indicación explícita de la pregunta o preguntas en las que trabajó cada integrante. Es indispensable que se detalle la contribución individual de cada miembro.
  5. Enlace al repositorio pùblico donde se encuentra alojado el código fuente del proyecto.
- **Nombre del fichero:** El archivo del documento debe ser nombrado siguiendo el formato `SemArq1_NombreEquipo`. Por ejemplo, si el equipo se llama “Los Arquitectos”, el fichero debe llamarse `SemArq1_LosArquitectos.pdf` (o la extensión correspondiente).
- **Documento:** Cada equipo entregrá un documento único con las respuestas a todas las preguntas planteadas en la sección anterior.

Uso de Fuentes y Citas Bibliográficas

Todo el contenido del documento debe ser original y estar debidamente fundamentado. En consecuencia, se establecen las siguientes normas de obligado cumplimiento:

- **Cita obligatoria de todas las fuentes:** Cualquier idea, concepto, definición, dato, figura o fragmento de texto que no sea de elaboración propia debe estar acompañado de su correspondiente cita bibliográfica en el cuerpo del documento. Esto aplica tanto a las obras listadas en la sección de *Bibliografía Recomendada* como a cualquier otra fuente externa (libros, artículos, sitios web, documentación técnica) que el equipo haya consultado y utilizado para la realización del seminario.
- **Formato de citas:** Se debe emplear un estilo de cita consistente a lo largo de todo el documento. Se recomienda el uso de un sistema de autor-año (por ejemplo, APA) o de

notas al pie, siempre que se aplique de manera uniforme. Todas las fuentes citadas en el texto deben aparecer en la sección de referencias bibliográficas final, y viceversa.

Declaración de Uso de Inteligencia Artificial

En coherencia con los principios de transparencia y honestidad académica, se establece lo siguiente respecto al uso de herramientas de inteligencia artificial (IA):

- **Obligatoriedad de declaración:** Si el equipo ha utilizado alguna herramienta de inteligencia artificial (como ChatGPT, Gemini, Claude, Copilot, u otras similares) para la elaboración del seminario, es obligatorio incluir un epígrafe independiente titulado **"Declaración de Uso de la Inteligencia Artificial"** al final del documento, después de la bibliografía.
- **Contenido de la declaración:** En dicho epígrafe se debe explicar de forma clara y detallada:
  1. **Qué herramienta(s) de IA** fue(ron) utilizada(s), especificando su nombre y versión cuando sea posible.
  2. **Con qué propósito** fue(ron) empleada(s) (por ejemplo: generación de ideas, redacción de borradores, corrección de estilo, traducción, generación de código LaTeX, elaboración de diagramas).
  3. **De qué forma** se integró el resultado en el documento final (por ejemplo: se utilizó como base para una sección que luego fue reescrita y verificada por el equipo, se empleó únicamente para mejorar la redacción, se usó para generar ejemplos de código que fueron analizados y adaptados, etc.).
- **Responsabilidad del contenido:** El uso de IA no exime al equipo de la responsabilidad sobre la totalidad del contenido entregado. Los estudiantes son los únicos responsables de verificar la veracidad, originalidad y calidad de la información generada, así como de asegurarse de que no se incurra en plagio o en el uso indebido de fuentes.
- **Ejemplo de epígrafe:** A modo de ilustración, la declaración podría tener la siguiente estructura:

  > **Declaración de Uso de la Inteligencia Artificial**
  > Para la elaboración de este seminario, el equipo utilizó la herramienta ChatGPT (versión GPT-4) con el propósito de generar ideas iniciales para la estructura del documento y mejorar la redacción de algunos párrafos. Los integrantes que hicieron uso de esta herramienta fueron [Nombre 1] y [Nombre 2]. Todo el contenido generado fue revisado, verificado y adaptado por el equipo, quien asume la responsabilidad final del documento.

Entrega y Repositorio

La entrega del seminario se realizará a través de dos canales obligatorios y complementarios:

- **Plataforma de evaluaciones:** El documento final debe ser subido a la sección "Evaluaciones" del grupo correspondiente a la asignatura **Ingeniería de Software** en Telegram.
- **Repositorio del proyecto:** El mismo documento, junto con cualquier material adicional (código fuente en LaTeX, diagramas o cualquier material complementario), debe estar alojado en el repositorio del proyecto del equipo. El enlace a dicho repositorio debe ser incluido en la página de presentación del documento, tal como se indicó anteriormente.

4. Bibliografía Recomendada

Para la preparación de este seminario y la resolución de las preguntas, se recomienda el estudio de la siguiente bibliografía.

- **Cockburn, A. (2005).** *Hexagonal Architecture (Ports and Adapters).* Consultado en: https://alistair.cockburn.us/hexagonal-architecture/
- **De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010).** *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador).* Microsoft Ibérica.
- **Martin, R. C. (2017).** *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.
- **Palermo, J. (2008).** *The Onion Architecture.* Consultado en: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/
- **Richards, M. (2015).** *Software Architecture Patterns.* O'Reilly Media.
- **Richards, M., & Ford, N. (2020).** *Fundamentals of Software Architecture: An Engineering Approach.* O'Reilly Media.